

How the pipeline actually works

Every stage a document passes through from upload to answer — the deterministic rules, the statistical fusion math, and the six places an LLM is ever called — traced directly from the current codebase, not from the design intent.

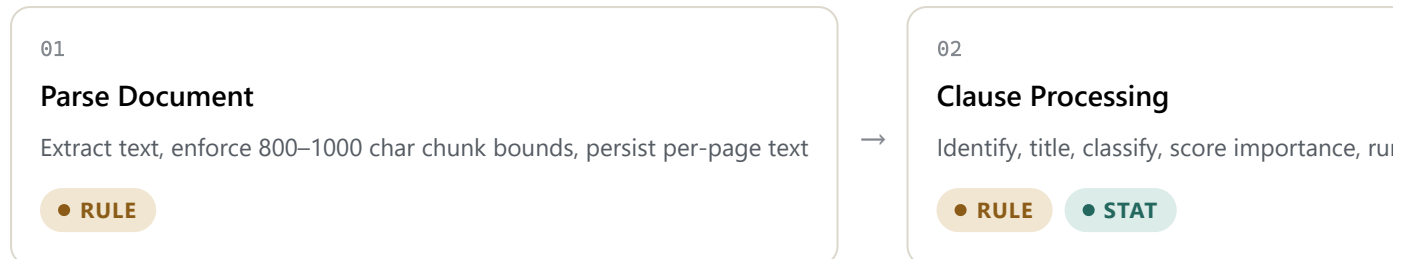
Orchestrated by `LangGraph` Retrieval via `ChromaDB` Records in `MongoDB` LLM: `Groq · llama-3.1-8b-instant`

6 pipeline stages	1 automatic LLM call / upload	5 on-demand-only LLM call sites	5 risk dimensions (LRSI)
~110 regex patterns, code + config	3× entropy-weighting applied		

- RULE-BASED** regex / fixed JSON tables, no model
- STATISTICAL** embeddings, entropy weighting, Jenks breaks
- LLM CALL** Groq round-trip, always single-clause or bounded-context

The ingestion pipeline

One LangGraph StateGraph, six nodes, a strictly linear edge sequence. Every node except the last runs with no LLM involved — most of the "intelligence" in this app is deterministic rule evaluation and statistical fusion, not model calls.



Standing architectural mandate: the LLM never receives a full document or multi-clause concatenation — only a single clause's text, a retrieved-chunk context block, or a small bounded stats summary (`services/prompt_builder.py`). Every prompt is built by one of three shared, bounded builders.

Parsing & chunking

● RULE

PDF (pdfplumber, with Tesseract OCR fallback for scanned pages), DOCX (python-docx), TXT, and image files all funnel into the same section-segmentation logic.

Section-heading detection

Four regex patterns, tried in order — first match wins. Each is capped at SECTION_HEADING_MAX_CHARS = 80 chars so body sentences can't misfire as headings.

PATTERN	REGEX	CATCHES
Numbered	^(?:section clause article part)\s\d+(?:\.\d+)*[:\-\s\.] ^\d+\.\s+[A-Z]	"Section 3.2 Foo"
Roman numeral	^(?:article\s+)?[IVXLCDM]{1,6}\.?\s*[-:.\s]*[A-Z]	"III. Termination "Article IV – Payment"
ALL-CAPS	^[A-Z][A-Z0-9 &,\-]{2,59}\$	"TERMINAT "GOVERNIN LAW"
Title Case	^[A-Z][a-zA-Z]*(?:\s+[A-Z][a-zA-Z]*){0,5}\$	"Governing Law", "Confidenti Obligations"

If regex segmentation yields ≤ 2 sections (parsing failure), the whole document falls back to RecursiveCharacterTextSplitter chunks instead — every document ends up sectioned one way or another.

Chunk bound enforcement

max_chars = 1000

chunk_overlap = 150

splitter = RecursiveCharacterTextSplitter

Any regex-derived section exceeding 1000 chars is re-split via LangChain's splitter (enforce_chunk_bounds); sections under the bound pass through untouched. This runs as a second Stage-1 pass over every raw section, closing the gap where the heading regex alone could produce an arbitrarily long section.

Clause identification & document typing ● RULE

Text is split into paragraph blocks (`\n\n` or a numbered-line boundary), each block scored against every registered clause type, and the best match kept if it clears a confidence floor.

Clause-type confidence formula

```
RULE_ENGINE.DETECT_CLAUSE_TYPE()
```

```
score = min( 1.0, round( 0.25 · regex_hit + 0.10 · min(keyword_hits, 5) + heading
```

```
    regex_hit      = 1 if the clause type's regex matches, else 0
```

```
    keyword_hits   = count of the type's keyword-vocabulary hits, capped at 5
```

```
    heading_bonus  = 0.15 if the type's name appears in the block's first line, else
```

clause accepted if score $\geq$ **0.30** (MIN_CONFIDENCE)

9 registered types (rules/clause_rules.json)

Rule content — the keyword lists and regex per type — lives in `rules/clause_rules.json`, not in Python: Termination, Liability, Confidentiality, Arbitration, Payment, Indemnity, Compliance, Jurisdiction, Force Majeure. Anything that doesn't clear the bar for a specific type falls through as "General."

Clause titling

The clause *type* (e.g. "Confidentiality") is a category for grouping/analytics only — never shown as the title, since one NDA routinely has six Confidentiality clauses.

`generate_clause_title()` checks a curated subtopic keyword table per type first (e.g. "shall not disclose" → "Non-Disclosure Obligations"), falls back to a generic per-type title, then to a Title-Cased first sentence for unrecognized types. Duplicate titles within one document get a numeric suffix.

Document-type classification — two tiers

`classify_document_type()` is the original, unchanged function the persisted `document_type` field depends on: highest total regex-match count across **21 registered types** ($\approx$ 90 patterns total), or "Unknown Document" if nothing clears `MIN_CONFIDENT_SCORE = 1`.

DOCUMENT_CLASSIFIER.CLASSIFY_DOCUMENT_TYPE_RANKED() — STAGE 0 OF THE AUTHENTICITY ENGINE

```

margin_confidence = (best_score - second_score) / (best_score + second_score)
evidence_confidence(n) = n / (n + 1) ← Laplace / add-one smoothing
confidence = margin_confidence × evidence_confidence(best_score)

```

Both factors must be strong: an uncontested winner (margin ≈ 1) backed by only one stray keyword still reads as barely more than a coin flip. Confirmed on a real tied case — "Lessor/Lessee" vs "Landlord/Tenant" phrasing scoring equally — where the plain classifier picks one type with no hint of doubt, and the ranked version correctly reports 0.0 confidence.

TYPE	SAMPLE PATTERNS
Non-Disclosure Agreement (NDA)	<code>non[-]?disclosure agreement</code> , <code>disclosing party</code> , <code>receiving party</code>
Loan Agreement	<code>loan agreement</code> , <code>\blender\b</code> , <code>\bborrower\b</code> , <code>principal amount</code>
Insurance Policy	<code>insurance policy</code> , <code>\bpolicyholder\b</code> , <code>\bpremium\b</code> , <code>sum assured</code>
FIR	<code>first information report</code> , <code>\bfir no\b</code> , <code>police station</code> , <code>\bipc\b</code>
Court Order	<code>in the high court</code> , <code>supreme court</code> , <code>\bpetitioner\b</code> , <code>\brespondent\b</code>
... 16 more types	Employment (×2 variants), Service, Lease, Rental, Purchase, Sale Deed, Vendor, MoU, Legal Notice, Affidavit, Partnership, Privacy Policy, Terms & Conditions, Power of Attorney, Will

02a · agents/importance_agent.py

Clause importance scoring ● RULE

Tier base score, then additive modifiers for money, uncapped exposure, and penalty language; boilerplate sections are capped low regardless of type.

IMPORTANCE_AGENT.ASSESS_CLAUSE_IMPORTANCE()

```

base = 80 if clause_type ∈ {Liability, Indemnity, Termination, Jurisdiction}
      = 55 if clause_type ∈ {Payment, Confidentiality, Compliance}
      = 30 otherwise

```

```

score = base + 8·[has money] + 12·[has "uncapped"/"unlimited"] + 10·[has "penalt
score = min(score, 25) if boilerplate heading (preamble / notice / signature /
score = clamp(score, 0, 100)

```

score ≥ **75** → Critical

score ≥ **40** → Important

else → Informational

Four-dimension impact scoring RULE

Starts from a per-clause-type baseline (rules/impact_rules.json), then nudges Legal / Financial / Business / Compliance independently.

Clause Type	Legal	Financial	Business	Compliance
Liability	70	75	40	30
Indemnity	75	70	35	30
Termination	60	40	50	20
Compliance	50	30	30	75
Payment	30	70	50	20
Arbitration	55	25	20	15
Jurisdiction	55	15	15	25
Confidentiality	40	20	30	50
Force Majeure	35	30	55	20
General (default)	10	10	10	10

```
financial += 15    if money detected in clause text
compliance += 20   if text contains "regulation"/"statute"/"sanction"/"fcpa"/"com
business  += 20    if text contains "sla"/"deliverable"/"milestone"/"service leve
legal     += 15, financial += 10  if text contains "penalty"/"liquidated damage
all four  → clamp(0, 100)
```

Legal feature extraction ● RULE ● STAT

Produces the structured `LegalFeatureVector` every risk-engine dimension consumes — never a source of risk weight itself, only detection. spaCy (`en_core_web_sm`) handles dependency-parse obligation extraction and NER; dates, durations, and money stay on the proven regex extractors since spaCy's own DATE/MONEY entities are noisier on legal-formatted figures ("₹50,00,000", "Rs. 5 Lakh").

Obligation extraction — dependency parse, not regex

Walks the parse tree for every modal auxiliary (`tag_ == "MD"`): the modal's head is the governing verb, a subject is located via `nsubj/nsubjpass` (or inherited up the `conj` chain for coordinated verbs with an elided subject), and a `neg` child flips polarity.

shall / must / will

→ Obligation

may / might / could / should

→ Right

any modal + neg child

→ Prohibition

Evidence-based confidence — one formula, everywhere

UTILS/CONFIDENCE.PY — `EVIDENCE_CONFIDENCE(COUNT)`

$\text{confidence} = \text{count} / (\text{count} + 1)$

1 independent detector → 0.500

2 agreeing detectors → 0.667

3 agreeing detectors → 0.750

→ asymptotic toward, never reaching, 1.0

Replaced four separate hand-picked constants (0.85 dependency parse / 0.7 spaCy NER / 0.9 regex / 0.6 lexical) with one shared Laplace-smoothing curve, applied per extraction type against its own "second independent detector":

EXTRACTION	CORROBORATION CHECK
Obligations	Does the legacy <code>_OBLIGATION_RE</code> regex independently find a fuzzy-matching subject (SequenceMatcher ratio ≥ 0.5)?
Entities	Do spaCy NER and <code>PARTY_RE</code> character spans overlap?
Legal actions	Do the phrase lexicon and lemma lexicon both independently hit the same <code>action_type</code> ?
Dependencies	Is the referenced clause number unique in the document (unambiguous resolution)?
Document type (Stage 0)	Same formula reused for classification confidence — <code>services/document_classifier.py</code> imports it from <code>utils/</code> so <code>services/</code> never depends on <code>agents/</code> .

Financial term parsing

`lakh = ×100,000`

`crore = ×10,000,000`

`million = ×1,000,000`

`billion = ×1,000,000,000`

Cap detection is a two-list keyword check on the clause text: `CAP_WORDS = {cap, capped, limited to, maximum of, not to exceed}` vs `UNCAP_WORDS = {uncapped, unlimited, without limit, no limitation}` — the qualitative half of the Financial dimension's signal (see below); the raw amount is only ever the log-scaled magnitude, never multiplied by an "uncapped" bonus.

Legal Risk Severity Index (LRSI) • STATISTICAL

The full replacement for the old fixed keyword-point scorer (deleted outright — RISK_TABLE/score_risk_points/rules/risk_rules.json no longer exist). Every dimension gets two independent [0,1] signals per clause, fused per-document with weights derived from the score distribution actually observed — never a hand-typed number.

Financial

log-scaled largest monetary figure in the clause

Legal

indemnification / termination / waiver strength + obligation/prohibition count

Compliance

regulatory entities + compliance actions + jurisdiction presence

Operational

deadline + dependency count

Ambiguity

weak/hedging-modal ratio (feature) · embedding-outlier score (semantic)

Per-dimension raw feature signals (F_d)

```

F_financial(c)    = log1p( max( amount : amount ∈ clause's non-percentage financia
F_legal(c)        = Σ confidence(a)  for legal_actions a ∈ {termination, indemni
                                assignment, limitatio
                                + Σ confidence(o) for obligations o with polarity ∈ {OBLIGATION
F_compliance(c)  = Σ confidence(e) for entities e ∈ {LAW, ORG_REGULATOR, STATUTE,
                                + Σ confidence(a) for actions a ∈ {compliance, regulatory_filin
                                + 1.0 if jurisdiction detected, else 0
F_operational(c) = count(deadlines) + count(dependencies)
F_ambiguity(c)   = weak_modals / total_modals    (0.5 if no modal detected at all)

```

Financial / Legal / Compliance / Operational raw values are then **percentile-normalized within the document's own clauses** (`scipy.stats.rankdata`) — the same raw obligation count means something different in a 4-clause NDA than a 60-clause MSA, so nothing is compared against a fixed threshold. Ambiguity's ratio is already bounded [0,1] and skips this step.

Semantic signal (E_d) — prototype similarity

Financial / Legal / Compliance / Operational: max cosine similarity between the clause's embedding and that dimension's natural-language prototype sentences (`risk_engine/prototypes.json`, 3 per dimension, 12 total) — so "Liability shall be without limit" and "There is no cap on damages" score similarly despite sharing no substring.

DIMENSION	EXAMPLE PROTOTYPE SENTENCE
Financial	"This clause creates unlimited or uncapped financial exposure for a party."
Legal	"This clause allows termination without cause or without notice."
Compliance	"This clause requires adherence to sanctions or anti-corruption regulations."
Operational	"This clause creates dependencies on multiple other obligations or milestones."

AMBIGUITY'S E_d — EMBEDDING-SPACE OUTLIER SCORE (NO PROTOTYPE SENTENCE EXISTS FOR "VAGUE")

$E_{\text{ambiguity}}(c) = 1 - \text{cosine_similarity}(\text{embedding}(c), \text{mean}(\text{all clause embeddings}))$

Fusion — feature $\times$ semantic, per dimension

RISK_ENGINE/FUSION.PY — FUSE_SIGNAL()

$S_d(c) = \text{clamp}(\alpha_d \cdot F_d(c) + (1 - \alpha_d) \cdot E_d(c), 0, 1)$

α_d (feature/semantic trust) is not fixed at 0.5 — it's **dynamic_alpha()**: `entropy_weights()` applied one level down, to a 2-column $[F_d, E_d]$ matrix across the document's clauses. Whichever branch varies more across this specific document earns more trust automatically.

Entropy Weight Method — where every weight in this engine comes from

RISK_ENGINE/FUSION.PY — ENTROPY_WEIGHTS() (SHANNON ENTROPY, ZELNY 1982)

$p_{ij} = (S_{ij} + \epsilon) / \sum_i (S_{ij} + \epsilon)$

$k = 1 / \ln(n)$

$e_j = -k \cdot \sum_i p_{ij} \cdot \ln(p_{ij})$

$d_j = (1 - e_j) + \epsilon$

row-normalized score matrix

n = clause count; undefined

entropy of dimension j

degree of divergence ("in

```

w_j      = d_j / Σ_j d_j          raw entropy weight, Σw_j

λ        = n / (n + n₀)          n₀ = 10 (shrinkage_n0)
w_final  = λ · w_raw + (1 - λ) · (1/m)    m = 5 dimensions

```

A dimension whose scores vary a lot across the document's clauses carries more discriminative information and is weighted up automatically; one that scores nearly the same everywhere is weighted down. **Small-n shrinkage** exists because a real bug surfaced it: raw EWM on a 3-clause test gave one dimension ~49% weight off a single outlier clause, ranking vague boilerplate above an unlimited-liability clause. At n=10 the blend is 50/50 raw-vs-uniform; it trusts the data more as clause count grows.

This exact entropy-weighting routine is reused **three times** at three granularities: across the 5 risk dimensions (`dimension_weights`), within each dimension's F/E fusion (`dimension_alphas`, above), and across the three ingredients of clause confidence (`confidence_weights`, below). One algorithm, three applications — the project's core recurring design idiom.

The index itself

```

RISK_ENGINE/FUSION.PY — LRSI_SCORES()

LRSI(c) = 100 · Σ_d w_d · S_d(c)          0-100, per clause
contribution_d(c) = round( w_d · S_d(c) · 100, 2 )    exact additive attribution

```

Confidence score — a third entropy-weighted fusion

```

RISK_ENGINE/EXPLAIN.PY — COMPUTE_CONFIDENCE_SCORES()

agreement      = 1 - mean_d | F_d(c) - E_d(c) |          signals disagree
feature_confidence = mean(confidence of every extracted feature)  0.5 default if
margin         = top semantic signal - second-highest semantic signal

confidence(c) = 100 · Σ [ entropy_weight(agreement, feature_confidence, margin) ×

```

Replaced a fixed 0.4/0.3/0.3 split — observed real weights on a 3-clause sample: 0.291 / 0.261 / 0.447, confirming the components genuinely aren't equally informative document to document.

Classification thresholds — Jenks Natural Breaks, not fixed cuts

Fisher-Jenks exact dynamic-programming algorithm (1958) minimizes within-class variance over this installation's own historical LRSI distribution — the same "goodness of variance fit" criterion used for choropleth map class breaks.

cold-start clause cuts: 35 / 70

cold-start document cuts: 35 / 60 / 80

MIN_REFERENCE_SIZE = 30

MAX_SAMPLE_SIZE = 1500

Below 30 reference scores, falls back to the fixed cold-start cuts (same philosophy as the shrinkage above: a principled fixed default beats an unstable data-derived one on too little data). Above it, Jenks computes real breaks from up to 1500 sampled historical scores, cached per-process via `ThresholdRegistry`, refreshable without rebuilding the engine.

Document-level aggregate — Gini-adjusted, not a step function

`RISK_ENGINE/FUSION.PY — DOCUMENT_RISK_SCORE()`

`Gini(LRSI values) = ( n + 1 - 2 · Σ(cumsum) / cumsum[last] ) / n` clamped [0

`document_risk_score = min( 100, mean(LRSI) · (1 + λ · Gini) )` λ = 0.15

Replaces the old "+10 points if >30% of clauses are High risk" step function with a continuous inequality statistic: a document where risk concentrates in a few severe clauses scores higher than one with the same average spread evenly — no arbitrary percentage threshold.

Tier-A audit — what's still a disclosed constant

A full 2026-07-19 audit found 10 remaining hardcoded values and resolved 5 into data-derived formulas (shown above). What's left, deliberately:

CONSTANT	VALUE	WHY IT STAYS FIXED
ε (epsilon)	1e-4	Numerical-stability floor for entropy $\log(0)$, not a belief about importance
shrinkage_n0	10	Single disclosed, ablatable global constant — the sample size where the blend is 50/50
concentration_λ	0.15	Same kind of disclosed constant, controlling Gini's influence on the document score

Dropped, not replaced: the Financial dimension's old 1.3× "uncapped" multiplier was removed outright rather than re-derived — the qualitative uncapped/unlimited signal is already captured by the semantic branch (`E_financial`), so a second multiplier on the feature branch was double-counting, not missing information.

Authenticity check RULE

Deliberately independent from legal risk scoring — a fabricated document with bland, low-risk-sounding clauses would score "low risk" and still be fake, so the two scores must never be blended. `contradiction_agent.py` never reads `risk_level` either, same discipline.

Live scorer — flat deductions from 100

```
authenticity_score = max( 0, 100 - Σ deductions_applied )
```

CHECK	DEDUCTION	DETECTION
Missing signatures	-20	<code>\b(signature signed /s/ authorized representative)\b</code> in last 20% of text
Missing party names	-20	<code>PARTY_RE</code> or <code>\bBETWEEN\b.{0,200}? \bAND\b</code>
Missing execution section	-15	<code>IN\s+WITNESS\s+WHEREOF</code> in last 25% of text
Missing dates	-15	zero <code>extract_dates()</code> matches anywhere
Unrealistic formatting	-15	< 500 chars total, or ≤ 1 section
Broken numbering	-10	decreasing or repeated leading section numbers
Missing governing law	-10	no Jurisdiction-clause keyword anywhere
Dangling references	-10	a "Section N" reference with no matching section
Empty mandatory clauses	-10	neither Termination nor Jurisdiction clause detected
Duplicate clauses	-10	SequenceMatcher ratio > 0.9 between two clauses
Missing witnesses	-10	only for Deed / Affidavit / Will document types

score ≥ 70 → Authentic

score ≥ 40 → Suspicious

else → Highly Suspicious

One check is deliberately a warning, never a deduction: a PO-Box-only address with no nearby postal code (`_PO_BOX_RE` / `_POSTAL_CODE_RE`) — regex cannot verify a real-world address, so it's surfaced as a soft signal only.

In progress — Document Authenticity Index (DAI) redesign

• **STATISTICAL** A 7-factor entropy-weighted replacement, reusing the same `fusion.entropy_weights()` and Jenks-threshold machinery built for the risk engine — with its own separate reference corpus so authenticity and risk can never correlate by construction. **Factor 1 (Structure Validation) is built; Factors 2–7 and the live wiring are not — `agents/authenticity_agent.py` above still runs unmodified in production.**

AUTHENTICITY/STRUCTURE.PY — `ASSESS_STRUCTURE()` (BUILT, NOT YET WIRED)

```
score          = c · type_specific_score + (1 - c) · generic_minimal_score      c = S
confidence = 100 · evidence_confidence(sections_found) · (0.5 + 0.5·c)
```

A continuous blend, never a hard template switch — validated on a synthetic insurance policy: full template match scored 0.95; removing just the policy number dropped it to 0.84, a proportional hit instead of the old scheme's flat cliff.

Contradiction detection ● RULE ● LLM (OPTIONAL)

Six-step pipeline, rule-based steps first, an optional LLM semantic pass last, everything consolidated to issue-level via union-find before returning. On automatic ingestion `use_llm=False` — the LLM pass runs only once a user opens the Contradiction Detection page.

1. **Duplicate grouping** — by shared clause number or normalized title (title groups capped at `MAX_TITLE_GROUP_SIZE = 4` to avoid every same-type clause in a document looking "duplicated").
2. **Numeric conflicts** — label-anchored regex, classification-agnostic.
3. **Temporal / payment / jurisdiction conflicts** — keyword-gated date/money/state comparison.
4. **Obligation conflicts** — subject-fuzzy-match + negation flip detection.
5. **Optional LLM semantic pass** — top 12 embedding-similarity pairs the rules didn't already resolve.
6. **Consolidation** — union-find merges same-(type, topic) pairwise findings into one issue per connected component.

THE LABEL-ANCHORED NUMERIC-CONFLICT REGEX

```
\b(penalty|late fee|interest(?:\s+rate)?|security deposit|deposit|discount|commis
rent|liability cap|cap on liability|notice period|cure period|grace period|ann
service fee|subscription fee|management fee|processing fee)\b
[^\n%$]{0,25}?
(\d+(?:\.\d+)?\s?%|(?:USD|US\$|\$|₹|INR|Rs\.?|€|£)\s?[\d,]+(?:\.\d+)?)
```

SequenceMatcher exact-repeat skip: ≥ 0.97

obligation same-subject: ≥ 0.6

opposite-statement same-target: ≥ 0.55

embedding candidate floor: ≥ 0.35

max LLM pairs checked: 12

max contradictions returned: 20

The embedding floor is deliberately generous (recall over precision) — a pair that passes it but isn't a real contradiction just costs one extra cheap LLM call, not a wrong answer shown to the user; the LLM's own "be conservative" system instruction is the real precision gate.

Knowledge graph & clause dependencies RULE

Regex-only entity/relationship extraction into a bounded NetworkX graph, plus explicit cross-reference and static domain-table dependency edges — none of this calls an LLM.

NAME	REGEX
PARTY_RE	([A-Z][A-Za-z0-9&.,\ -]{2,60}?)\s*(\s*["'"]{2,40})["'"]\s*)
JURISDICTION_RE	(?:governing law governed by the laws of state of courts of)\s+(?:the\s+)?([A-Z][A-Za-z0-9&.,\ -]{2,60}?)
PENALTY_RE	\b(penalty liquidated damages late fee)\b[^\n]{0,60}

MAX_NODES = 25

MAX_EDGES = 40

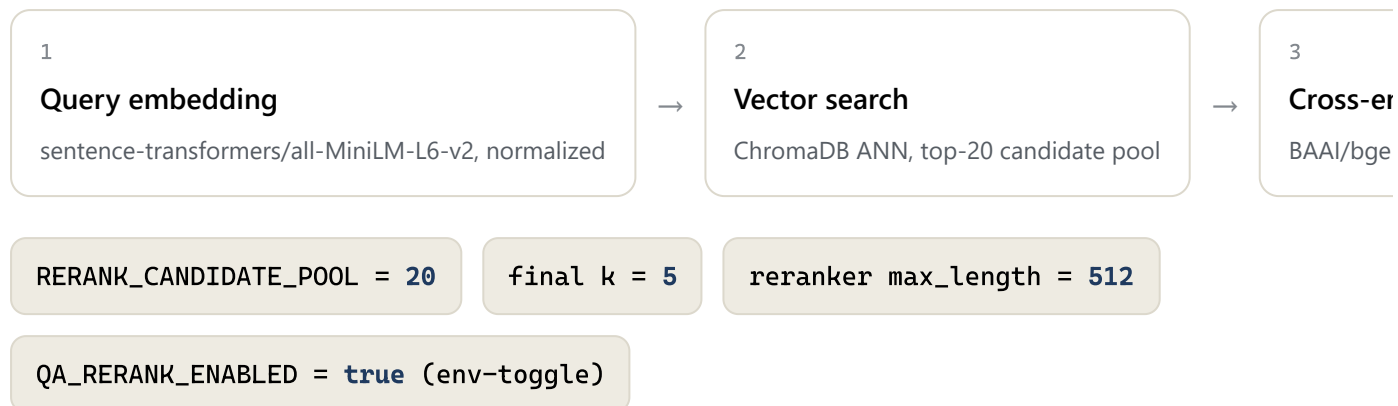
Dependency edges come from two independent sources: explicit numbered cross-references in clause text (Section 5.2 → matched against another clause's leading number) and a static rules/dependency_rules.json domain table — e.g. Termination → triggers → Liability, Payment → subject_to → Compliance, 7 relations total.

RAG retrieval pipeline

• STATISTICAL

• LLM

Every module needing grounded context — Dashboard, Clause Analysis, Risk Analysis, Contradiction Detection, Comparison Center, Legal AI Chat — shares this one retrieval path. Deliberately minimal: no intent-based metadata filtering, no BM25/RRF fusion in the live path (`utils/hybrid_search.py` exists but is currently unused — dead code), no sentence-level compression.



Three shared, bounded prompt builders in `services/prompt_builder.py` are the *only* place LLM-facing prompt strings get assembled — no call site builds its own ad hoc string:

build_clause_prompt

Single clause: heading + full text + instructions

build_context_block

Retrieved-chunk citation blocks, tagged by doc/section

build_summary_prompt

Small bounded stats + pre-truncated sample lines — never full clause text

LLM layer

Every structured LLM call goes through one function, `invoke_llm_structured()`: builds a concrete example-JSON prompt from the Pydantic schema (small instruct models echo raw JSON-Schema vocabulary back verbatim otherwise), parses the response, and retries once with a correction prompt on failure.

TOKEN-BUDGET ENFORCEMENT — EVERY PROMPT IS PROVABLY BOUNDED BEFORE IT'S SENT

chars_per_token ≈ 3.5 (conservative estimate for dense legal/English text)

budget_tokens = GROQ_TPM_LIMIT - MAX_COMPLETION_TOKENS - estimate_tokens(syst

budget_chars = max(budget_tokens × 3.5, 500)

if len(user_prompt) > budget_chars: truncate + append "[... input truncated ...]"

GROQ_TPM_LIMIT = 6000 tokens/min

MAX_COMPLETION_TOKENS = 2048

model = llama-3.1-8b-instant

safety_margin = 500 tokens

Every LLM call site in the app

CALL SITE	TRIGGER	SCOPE
<code>audit_agent.perform_macro_audit</code>	● AUTOMATIC — every upload	Bounded stats summary (clause counts, 3 sample high-risk snippets ≤200 chars each)
<code>qa_agent.answer_legal_question</code>	On-demand — Legal AI Chat	Top-5 retrieved/reranked chunks only
<code>analyzer_agent.analyze_clause_risk_with_llm</code>	On-demand — one clause at a time	Single clause text
<code>risk_scoring_agent.generate_risk_mitigation</code>	On-demand — "Simplify Risk" button	Single clause + its risk explanation
<code>risk_scoring_agent.generate_improved_clause</code>	On-demand — "Generate Improved Clause"	Single clause + mitigation breakdown
<code>contradiction_agent._llm_semantic_check</code>	On-demand — first Contradiction Detection page view	One clause pair at a time, ≤800 chars each

That's the entire LLM surface area of the application: **one** automatic call per document upload, five more only ever fired by an explicit user action, and every single one bounded to a clause, a clause pair, or a small stats block.

database/ + vectorstore/

Data layer

MongoDB — 10 collections

users · documents · clauses · contradictions · clause_versions · audit_logs · pages
· entities · relationships · retrieval_history

Auto-incrementing integer IDs via a **counters** collection (**find_one_and_update** with **\$inc**) rather than Mongo's default ObjectId, so IDs stay human-referenceable across the UI.

ChromaDB — vector store

Clause-level embeddings, filterable by **document_id** metadata (a field that was silently defaulting to "unknown_doc" until a recent orchestrator fix). Backs every retrieval call in the RAG pipeline above.

Normalized collections added alongside the retrieval/graph/authenticity work: **pages** (Stage-1 raw per-page text, PDF only), **entities/relationships** (persisted knowledge-graph output — previously computed and thrown away every page load), and **retrieval_history** (one row per QA query).

Regex appendix ● RULE

Every hand-authored extraction/detection pattern in the Python codebase (config-driven vocabularies — 90 document-type patterns, 9 clause-type regexes, 7 structure templates — are covered above in their own sections and summarized at the end of this table).

FILE	NAME	PATTERN
rule_engine.py	_MONEY_RE	(?:USD US\$ \\$ ₹ INR Rs\.? € £)\s?[\d,]+(?:\.
rule_engine.py	_DATE_RE	\b(?:\d{1,2}(?:st nd rd th)?\s+day\s+of\s+)?(?:
rule_engine.py	_DURATION_RE	\b(\d+)\s?(day days month months year years we
rule_engine.py	_SECTION_REF_RE	\b(?:Section Clause Article Paragraph)\s+\d+(?:
rule_engine.py	_OBLIGATION_RE	\b([A-Z][A-Za-z&.\]{2,40})\s+(shall(?:\s+not)?
rule_engine.py	_CLAUSE_NUMBER_RE	\b(?:Section Clause Article Paragraph)\s+(\d+(?:
rule_engine.py	_LEADING_NUMBER_RE	^\s*(\d+(?:\.\d+)*)\s*[\.\.]:]\s*
rule_engine.py	_TITLE_PREFIX_RE	^(section clause article paragraph)\s+\d+(?:\.
knowledge_graph_agent.py	PARTY_RE	([A-Z][A-Za-z0-9&.\-]{2,60}?)\s*(\s*"")(^[^
knowledge_graph_agent.py	JURISDICTION_RE	(?:governing law governed by the laws of state
knowledge_graph_agent.py	PENALTY_RE	\b(penalty liquidated damages late fee)\b[^\n
authenticity_agent.py	_SIGNATURE_RE	\b(signature signed /s/ authorized representat
authenticity_agent.py	_EXECUTION_RE	IN\s+WITNESS\s+WHEREOF
authenticity_agent.py	_WITNESS_RE	\bwitness(es)?\b
authenticity_agent.py	_BETWEEN_AND_RE	\bBETWEEN\b.{0,200}?\bAND\b
authenticity_agent.py	_PO_BOX_RE	P\.?0\.\?\s*Box\s*\d+
authenticity_agent.py	_POSTAL_CODE_RE	\b\d{5,6}\b
contradiction_agent.py	_LABELED_NUMBER_RE	see Contradictions §, label-anchored numeric/mon
dependency_agent.py	_LEADING_NUM_RE / _REF_NUM_RE	^\d+(?:\.\d+)* / (\d+(?:\.\d+)*
parser_agent.py	SECTION_PATTERNS ×4	see Parsing §, numbered / roman / ALL-CAPS / Title

Config-driven: clause types

`rules/clause_rules.json`

9 clause types, 1 regex + keyword list each

Config-driven: document types

`document_classifier.py`

21 types, ~90 regex patterns total

Config-driven: structure templates

`document_structure_rules.json`

7 templates (+ generic-minimal fallback), 4–7 sections each

Formula & threshold quick-reference

NAME	VALUE / FORMULA	WHERE
Chunk size / overlap	1000 chars / 150 overlap	parser_agent.py
Section heading max length	80 chars	parser_agent.py
Clause-type confidence	$0.25 \cdot \text{regex} + 0.10 \cdot \min(\text{kw}, 5) + 0.15 \cdot \text{heading}$	rule_engine.py
Clause acceptance floor	score ≥ 0.30	clause_identifier_agent.py
Doc-type confidence	$\text{margin_confidence} \times \text{evidence_confidence}(\text{best})$	document_classifier.py
Evidence confidence	$\text{count} / (\text{count} + 1)$	utils/confidence.py
Importance tiers	Critical 80 / Important 55 / else 30, $\pm 8/12/10$ modifiers	importance_agent.py
Importance classification	≥ 75 Critical, ≥ 40 Important, else Informational	importance_agent.py
Impact modifiers	+15/+20/+20/+15+10 per signal	impact_agent.py
LRSI fusion	$S_d = \text{clamp}(\alpha \cdot F_d + (1 - \alpha) \cdot E_d, 0, 1)$	risk_engine/fusion.py
LRSI index	$100 \cdot \sum w_d \cdot S_d(c)$	risk_engine/fusion.py
Entropy weights	Shannon EWM, $\lambda = n / (n + 10)$ shrinkage	risk_engine/fusion.py
Clause risk cuts (cold-start)	35 / 70	risk_engine/thresholds.py
Document risk cuts (cold-start)	35 / 60 / 80	risk_engine/thresholds.py
Jenks min reference size	30	risk_engine/thresholds.py
Gini-adjusted document score	$\min(100, \text{mean}(\text{LRSI}) \cdot (1 + 0.15 \cdot \text{Gini}))$	risk_engine/fusion.py
Authenticity deductions	$100 - \sum (10 - 20 \text{ per failed check})$	authenticity_agent.py
Authenticity classification	≥ 70 Authentic, ≥ 40 Suspicious, else Highly Suspicious	authenticity_agent.py
DAI structure blend (in progress)	$c \cdot \text{type_score} + (1 - c) \cdot \text{generic_score}$	authenticity/structure.py

NAME	VALUE / FORMULA	WHERE
Contradiction similarity gates	0.97 / 0.6 / 0.55 / 0.35	contradiction_agent.py
Retrieval pool → final k	20 → 5	retrieval_service.py
Groq token budget	6000 TPM – 2048 completion – system – 500	llm_client.py

Current state

Build status

- **Hybrid Explainable Risk Engine (LRSI)** — fully implemented and wired into the live pipeline. Old keyword-point scorer deleted outright.
- **Jenks thresholds, dynamic α , evidence-based confidence, entropy-weighted confidence formula, dropped Financial multiplier** — all 5 audit-remediation items complete.
- **Centralized RAG retrieval** — every analysis module shares one pipeline; no view calls the LLM client directly.
- **Document Authenticity Index (DAI)** — Stage 0 (ranked document-type classification) and Factor 1 (Structure Validation) built and tested; Factors 2–7 and live wiring not yet started. Production still runs the flat-deduction scorer.
- **Risk Heatmap (clause × dimension)** — data (dimension_breakdown) persisted and ready; no dashboard UI built yet.
- **Legal-action classification** — still a placeholder lexical lookup (fixed 0.6 baseline confidence pre-corroboration); zero-shot/fine-tuned classifier is the stated future upgrade.
- **BM25/RRF hybrid search** (utils/hybrid_search.py) — implemented but never imported; live QA path is vector search + rerank only.

Compiled directly from the current repository state (agents/, risk_engine/, authenticity/, services/, rules/, database/) rather than from design docs — every formula, threshold, and regex above is copied from the actual source. Re-verify against the code before citing a

specific number in a paper: constants like `shrinkage_n0` or the cold-start cut points are explicitly designed to be ablatable and may change.